

Separation Logic and the Aliasing Problem

Lani Quach

Why is heap verification difficult?

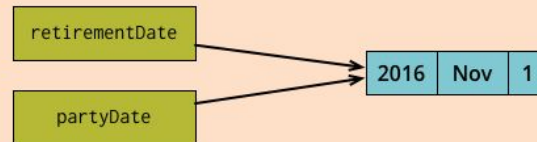
Aliasing: When multiple pointers refer to the same memory location, modifying through one silently affects the other.

- Consequences: use-after-free, null pointer dereferences, buffer overflows can go undetected
- Modern day programs rely on dynamic memory and pointer manipulation
- Need for formal reasoning tools

```
Date retirementDate = new Date(Date.parse("Tue 1 Nov 2016"))
```



```
Date partyDate = retirementDate;
```

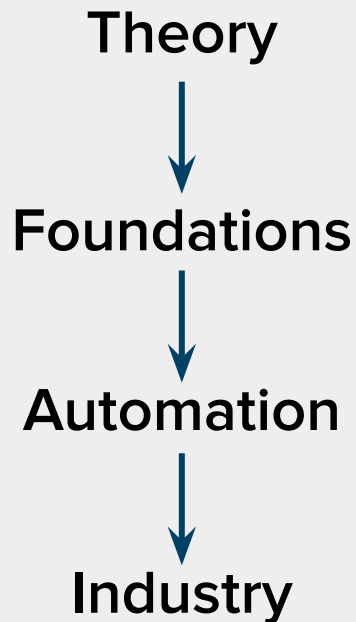


```
partyDate.setDate(5);
```



Paper Timeline

- 1969: Hoare — **Axiomatic basis for program verification**
- 2001: O'Hearn, Reynolds, Yang — **The Frame Rule**
- 2001: Ishtiaq & O'Hearn — **BI as Assertion Language**
- 2002: Reynolds — **Separation Logic Survey**
- 2005: Berdine, Calcagno, O'Hearn — **Smallfoot**
- 2009: Calcagno, Distefano, O'Hearn, Yang — **Bi-Abduction**



An Axiomatic Basis for Computer Programming (Hoare, 1969)

- No formal mathematical framework for proving program correctness

Hoare triple: $\{P\} C \{Q\}$

P is a precondition, C is a command, Q is a postcondition

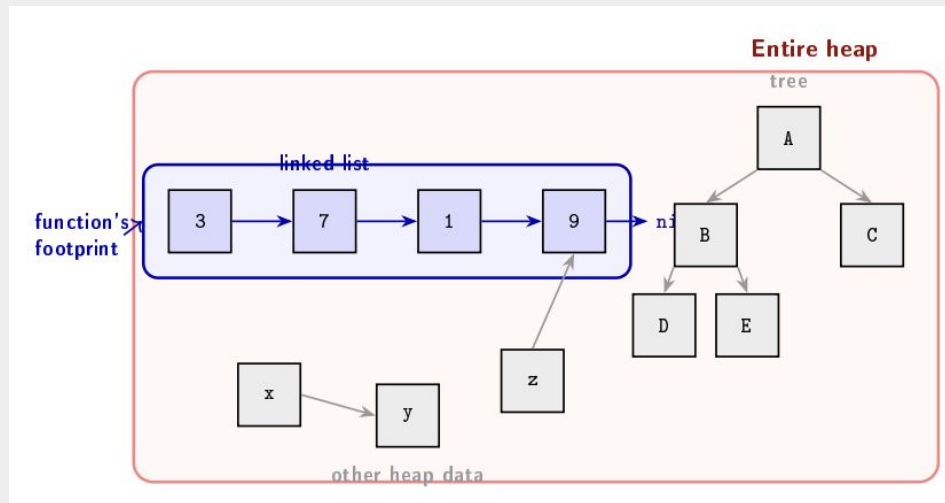
- Inference rules:
 - Assignment axiom: $\{P[e/x]\} x := e \{P\}$
 - Composition, Consequence, While (with loop invariants)
- Proofs mirror program structure - compositional

Hoare Logic: Evaluation & Limitations

- Evaluation: division algorithm proof using all four rules
- Limitation: assignment axiom assumes variable independence
 - $*x = 5$ may change $*y \rightarrow$ axiom breaks
- No support for heap allocation, deallocation, or recursive data structures

Local Reasoning About Programs That Alter Data Structures (O'Hearn, Reynolds, Yang, 2001)

- To verify a function, needed to specify the entire heap in the pre/postcondition
- Modular verification infeasible
- Need to be able to verify each component independently and compose results



The Frame Rule

if $\{P\} C \{Q\}$ holds and R describes heap disjoint from what C accesses,
then $\{P * R\} C \{Q * R\}$, provided C does not modify free variables of R

- **Seperating Conjunction** ($P*R$): “ P and R hold on separate, non-overlapping parts of the heap”
- **Small axioms:** specify only the footprint
 - Ex: the mutation axiom $\{E \mapsto -\} [E] := F \{E \mapsto F\}$ talks about only the single cell at address E
- Everything else automatically preserved

$E \mapsto F$ means address
 E holds value F

Frame Rule: Evaluation & Limitations

- **Paper proofs:** CopyTree, difference-linked list ops

$$\frac{\{\text{tree } \tau_1 i\} \text{CopyTree}(i; i') \{(\text{tree } \tau_1 i) * (\text{tree } \tau_1 i')\}}{\begin{array}{l} \{\tau \doteq (\tau_1, \tau_2) * (p \mapsto i, j) * (\text{tree } \tau_1 i) * (\text{tree } \tau_2 j)\} \\ \text{CopyTree}(i; i') \\ \{\tau \doteq (\tau_1, \tau_2) * (p \mapsto i, j) * (\text{tree } \tau_1 i) * (\text{tree } \tau_2 j) * (\text{tree } \tau_1 i')\} \end{array}}$$

- Frame rule shown complete (with consequence + aux vars)
- Limitation: choosing the correct frame requires human insight
- Automation identified as future work

BI as Assertion Language (Ishtiaq & O'Hearn, 2001)

- * lacked connection to established logic — completeness unknown
- **Separation logic $\subset$ logic of bunched implications (BI)**
 - BI (bunched implications) is a substructural logic with two families of connectives: additive ($\wedge$, $\rightarrow$) and multiplicative ($*$, $-*$)
 - Heap model = concrete resource semantics

Magic wand: $P -* Q$

"Add disjoint heap satisfying $P \rightarrow Q$ holds on combined"

- Classical variant needed for reasoning about dispose

BI: Evaluation & Limitations

- Proved soundness + completeness (Theorems 4, 5)
- $(x \mapsto 1,2) * (x \mapsto 1,2) = \text{always false}$ vs. $\wedge$ version = satisfiable
 - Contraction fails $\rightarrow$ substructural logic
- Limitations: no automation, magic wand hard to use in practice (Smallfoot/Infer avoid it), first-order only (no quantification over predicates or assertions)

Reynolds Survey (2002)

- Consolidated separation logic:
 - $P * Q, x \mapsto v$, emp, magic wand
 - Inductive predicates: lists, trees, doubly-linked lists, DAGs, xor-lists
- Proof rules: local, global, backwards forms
- Frame rule + recursive procedures + iterated * for arrays

Reynolds Survey: Evaluation & Limitations

- Paper proofs (increasing difficulty):
 - List reversal, deletion, tree/DAG copy, cyclic buffer
 - Schorr-Waite graph marking (previously defeated verification)
- All manual, sequential only, no automation

Smallfoot (Berdine, Calcagno, O'Hearn, 2005)

- Problem: all prior work = manual proofs
- Solution: automatic forward symbolic execution
 - Symbolic heap state as $\Pi \wedge \Sigma$
 - Applies proof rules at each program step
- Key innovation: automatic frame inference at procedure calls

```
[(p ↦ l: x, r: y) * tree(x) * tree(y)]  
  i := p → l; j := p → r;  
[(p ↦ l: i, r: j) * tree(i) * tree(j)]  
  disp_tree(i);  
[(p ↦ l: i, r: j) * tree(j)]  
  disp_tree(j);  
[p ↦ l: i, r: j]  
  dispose(p);  
[emp]
```

Smallfoot: Evaluation & Limitations

- Verified: tree disposal, tree copy, list append, memory manager, pointer-transfer buffer, etc.
- First automated separation logic tool
- Limitations:
 - still needs user-provided loop invariants + pre/postconditions
 - fixed predicates only
 - no fractional permissions

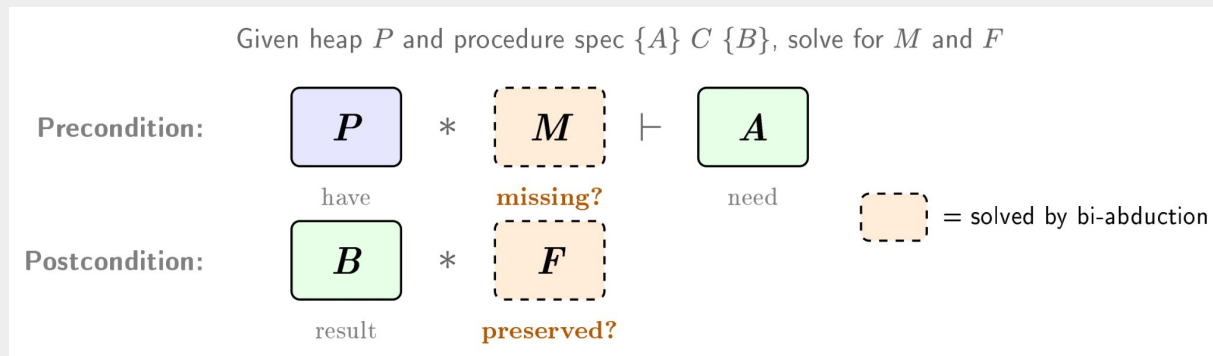
Bi-Abduction (Calcagno et al., 2009)

- Smallfoot needs complete annotations, doesn't scale
- Industry devs won't write formal specs
- Goal: zero-annotation specification inference

```
1  lst_nd* p(lst_nd *y) { // Inferred Pre: list(y)
2    lst_nd *x;
3    x=malloc(sizeof(lst_nd)); x->tail = 0;
4    merge(x,y); // Obtained Post: list(x)
5    return(x);
6  } // Inferred Post: list(ret)
7  void merge(lst_nd *x,lst_nd *y){//SUMMARY ONLY
8    // Given Pre: list(x) * list(y)
9    // Given Post: list(x)
10 }
```

Bi-Abduction: Core Idea

- Given: current heap P , call spec $\{A\} C \{B\}$
- Solve simultaneously:
 - Anti-frame M : $P * M \vdash A$ (what's missing?)
 - Frame F : postcondition $B * F$ (what's preserved?)



Bi-Abduction: Proof Rules

- Key rules:
 - $\mapsto$ -match: consume matching points-to facts
 - lseg-right/left: unfold list segments
 - remove: eliminate empty-heap predicates
 - missing: unmatched predicate $\rightarrow$ move to anti-frame

$$\begin{array}{c}
 \frac{(\Pi \wedge \Sigma) * [M] \triangleright H \quad \Pi \wedge \Sigma \vdash \exists \vec{a}. \Pi' \quad \Pi \wedge \Pi' \wedge \text{emp} \vdash \Sigma'}{(\Pi \wedge \Sigma) * [M] \triangleright H * (\exists \vec{a}. \Pi' \wedge \Sigma')} \text{remove} \\
 \\
 \frac{(E_0 = E_1 \wedge \Delta) * [M] \triangleright \exists \vec{b}. \Delta'}{\Delta * E \mapsto E_0 * [\exists \vec{a}. E_0 = E_1 \wedge M] \triangleright \exists \vec{a} \vec{b}. \Delta' * E \mapsto E_1} \mapsto\text{-match} \\
 \text{(where } \vec{b} \cap \text{FreeLVar}(E_1) = \emptyset \text{)} \\
 \\
 \frac{\Delta * [M] \triangleright \exists \vec{a}. \Delta' * \text{lseg}(E_0, E_1)}{\Delta * B(E, E_0) * [M] \triangleright \exists \vec{a}. \Delta' * \text{lseg}(E, E_1)} \text{lseg-right} \\
 \text{(where } B(E, E_0) \text{ is } E \mapsto E_0 \text{ or } \text{lseg}(E, E_0) \text{)} \\
 \\
 \frac{(E \neq E_0 \wedge \Delta * \text{lseg}(a, E_0)) * [M] \triangleright \exists \vec{b}. \Delta'}{\Delta * \text{lseg}(E, E_0) * [E \neq E_0 \wedge M] \triangleright \exists \vec{a} \vec{b}. \Delta' * E \mapsto a} \text{lseg-left} \\
 \\
 \frac{}{(\Pi \wedge \text{emp}) * [\exists \vec{a}. \Pi' \wedge \text{emp}] \triangleright \exists \vec{a}. \Pi' \wedge \text{emp}} \text{base-emp} \\
 \\
 \frac{}{\Delta * [\exists \vec{a}. \Pi \wedge \text{emp}] \triangleright \exists \vec{a}. \Pi \wedge \text{true}} \text{base-true} \\
 \\
 \frac{\Delta * [M] \triangleright \Delta' \quad \Delta * \exists \vec{a}. B(E, E') \not\vdash \text{false}}{\Delta * [M * \exists \vec{a}. B(E, E')] \triangleright \Delta' * \exists \vec{a}. B(E, E')} \text{missing} \\
 \text{(where } B(E, E') \text{ is } E \mapsto E' \text{ or } \text{lseg}(E, E') \text{)}
 \end{array}$$

Figure 1. Proof Rules for Abductive Inference

Bi-Abduction: Algorithms

- BiAbd (Alg 2): Abduce $\rightarrow$ find M, then Frame $\rightarrow$ find F
- Compositional analysis (Alg 3): bottom-up by call graph
 - InferPre (Alg 4) $\rightarrow$ discover preconditions
 - InferSpec (Alg 5) $\rightarrow$ compute postconditions
- Output: $\{P\} f(\bar{x}) \{Q_1 \vee \dots \vee Q_n\}$ per procedure
- No annotations, no whole-program analysis

Bi-Abduction: Evaluation

- SpacInvader Abductor tool
- IEEE 1394 firewire driver: 121 procedures, consistent specs
- 8 cores: Linux analysis 115 min → 29 min

Program	MLOC	Num. Procs	Proven Procs	Procs Coverage %	Time (1)	Time (8)
Linux kernel 2.6.25.4	2.473	101330	59215	58.4	6869.09	1739.28
Gimp 2.4.6	0.708	15114	6364	42.1	3601.16	1067.60
OpenSSL 0.9.8g	0.214	4818	2967	61.6	605.36	446.60
Sendmail 8.14.3	0.108	684	353	51.6	184.50	184.83
Apache 2.2.8	0.102	1870	881	47.1	294.67	104.48
OpenSSH 5.0	0.073	1135	519	45.7	142.56	30.24
Spin 5.1.6	0.019	357	197	55.2	772.82	253.96

Table 1. Case Studies with Larges Programs (timeout=1s).

Bi-Abduction: Applications and Limitations

- **Facebook Infer:** deployed on FB, Instagram, WhatsApp
 - Analyzes diffs before code review, catches bugs before production
- Completeness traded for scalability → false positives/negatives
- No code pointers, no concurrency
- Memory safety only, not functional correctness



Summary

- Hoare logic → separation logic → Smallfoot → bi-abduction/Infer
- Key idea: * enables local reasoning, frame rule enables composition
- From pen-and-paper proofs → thousands of bugs caught/month in production

Future Work

- Complex data structures (doubly-linked lists, graphs, user-defined predicates)
- Concurrent verification at scale (Iris/Coq promising, not industrial yet)
- Precision vs. scalability tradeoff

References

- [1] J. Berdine, C. Calcagno, and P. W. O'Hearn. "Smallfoot: Modular Automatic Assertion Checking with Separation Logic." Proc. FMCO, 2005.
- [2] C. Calcagno, D. Distefano, P. W. O'Hearn, and H. Yang. "Compositional Shape Analysis by Means of Bi-Abduction." Journal of the ACM, 2009.
- [3] C. A. R. Hoare. "An Axiomatic Basis for Computer Programming." Communications of the ACM, 1969.
- [4] S. S. Ishtiaq and P. W. O'Hearn. "BI as an Assertion Language for Mutable Data Structures." Proc. POPL, 2001, pp. 14–26.
- [5] P. W. O'Hearn, J. C. Reynolds, and H. Yang. "Local Reasoning About Programs That Alter Data Structures." Proc. CSL, 2001.
- [6] J. C. Reynolds. "Separation Logic: A Logic for Shared Mutable Data Structures." Proc. IEEE LICS, 2002.

Questions?